Heaps: A Tree Shaped for Priority, Not Order

Programming and Data Structures — Week 8, Lecture 2

Dr. Innocent Nyalala

Objectives

By the end of this lecture, you will be able to state the heap property precisely and explain how it fundamentally differs from Week 7's binary search tree property; implement a min-heap backed by a plain array, including the arithmetic formulas relating a node's array index to its parent's and children's indices; trace both `heapify_up` and `heapify_down` completely on concrete numeric examples; and explain why a heap provides O(1) access to the minimum element specifically, while offering no efficient support at all for finding any other value.

Outline

Today covers a hospital-triage-queue analogy for priority-based rather than value-based ordering; the heap property, contrasted directly against Week 7's BST property; the array-based representation that stores a complete binary tree with no pointers at all; insertion via `heapify_up`, traced completely; extracting the minimum via `heapify_down`, traced completely; why a heap trades away general search capability for O(1) minimum access, and why this is a deliberate design choice, not an oversight; the MTech-track `heapify` algorithm building a heap from an unordered array in O(n); an in-class quiz; and a summary.

The hospital-triage-queue analogy

An emergency room does not treat patients strictly in arrival order — the FIFO discipline of Week 3's queue would be dangerous, since a life-threatening case arriving after a minor one must be treated first. But maintaining patients in a fully sorted line by urgency is also wasteful: Week 7's balanced BST could do this, but a full BST provides far more capability — arbitrary search, floor and ceiling queries, full ordering — than triage actually needs. What triage genuinely requires is fast access to "who is the single most urgent patient right now," and fast insertion of new patients, without needing every other patient to be in any particular fully sorted order relative to each other. A heap provides exactly this narrower, cheaper guarantee.

The heap property, contrasted with the BST property

The heap property requires only that every parent be less than or equal to both of its children (for a min-heap; a max-heap reverses this), recursively, at every node. This is a strictly weaker guarantee than Week 7's BST property, which additionally required a strict left-versus- right ordering relationship. A heap makes no claim whatsoever about the relative order of a node's left and right subtrees, or about siblings anywhere in the tree — the only guarantee is the vertical parent-to-child relationship. This weaker guarantee is precisely why a heap cannot support efficient search, range queries, or floor/ceiling queries the way a BST can — but it is also exactly why heap operations can be made simpler and faster for the one thing a heap is built to do.

The array trick: no pointers at all

```
class MinHeap:
    def __init__(self):
        self.data = []    # a PLAIN array - no TreeNode, no left/right pointers

    def _parent(self, i):
        return (i - 1) // 2

    def _left_child(self, i):
        return 2 * i + 1

    def _right_child(self, i):
        return 2 * i + 2
```

A heap is always kept as a complete binary tree: every level is completely full except possibly the last, which fills strictly left to right with no gaps permitted anywhere. This completeness guarantee — which a general BST does not require — is precisely what allows the entire tree to be stored in one plain array, with no `left`/`right` pointers at all: a node at array index i has its parent at index $(i-1)//2$, and its children at indices $2i+1$ and $2i+2$. This is structurally the same idea as Week 1's row-major 2D array address formula — computing a position directly through arithmetic rather than following a stored reference — applied here to tree structure instead of grid structure.

Tracing the array-index formulas on a concrete heap

```
Array: [1, 3, 2, 7, 4, 5]
Index:  0  1  2  3  4  5

parent(3) = (3-1)//2 = 1   → arr[1]=3 is the parent of arr[3]=7. Check: 3<=7
```

```
left_child(1) = 2*1+1 = 3  → arr[3]=7 is the left child of arr[1]=3
right_child(1) = 2*1+2 = 4 → arr[4]=4 is the right child of arr[1]=3. Check: 3<=4
parent(0) = (0-1)//2 = -1//2 = -1  → index 0 (the root) has no parent, correctly
```

Verifying the formulas on a concrete 6-element array: computing `parent(3)` gives index 1, and checking `arr[1]=3` against `arr[3]=7` confirms the heap property holds there ($3 \leq 7$). Computing `left_child(1)` and `right_child(1)` correctly identifies indices 3 and 4 as node 1's children, and both satisfy the heap property relative to their parent. The root, index 0, correctly has no valid parent under this formula, confirming the arithmetic handles the tree's boundary correctly without any special-case code.

Insert: `heapify_up`, in code

```python
def insert(self, value):
    self.data.append(value)              # add at the very end (next open leaf position)
    self._heapify_up(len(self.data) - 1)

def _heapify_up(self, i):
    parent = self._parent(i)
    if i > 0 and self.data[i] < self.data[parent]:
        self.data[i], self.data[parent] = self.data[parent], self.data[i]
        self._heapify_up(parent)          # continue checking further up
```

Insertion appends the new value at the array's end, which is always the next available leaf position in a complete tree, automatically preserving completeness with no extra bookkeeping. `_heapify_up` then repeatedly compares the new element against its parent, swapping upward whenever the heap property is violated, continuing until either the property holds or the root is reached. This upward bubbling is bounded by the tree's height — and critically, because completeness is *always* maintained, a heap with n nodes always has height $O(\log n)$, guaranteed, with no equivalent to Week 7's plain-BST degeneration risk ever possible.

Tracing heapify_up completely, inserting 0

```
Before: [1, 3, 2, 7, 4, 5]      insert(0)
Step 1: append 0 → [1, 3, 2, 7, 4, 5, 0],  i=6
Step 2: parent(6) = (6-1)//2 = 2.  arr[6]=0 < arr[2]=2 → swap
        → [1, 3, 0, 7, 4, 5, 2],  i=2
Step 3: parent(2) = (2-1)//2 = 0.  arr[2]=0 < arr[0]=1 → swap
        → [0, 3, 1, 7, 4, 5, 2],  i=0
Step 4: i=0 is the root → stop
```

Tracing the insertion of 0 into the 6-element heap from earlier: it is appended at index 6, then compared against its parent at index 2 (value 2) — since $0 < 2$, they swap. The new position, index 2, is compared against its parent at index 0 (value 1) — since $0 < 1$, they swap again. Having reached the root, the process stops. The value 0, correctly the new minimum, has bubbled all the way to the root in exactly 2 swaps — $\log_2(7) \approx 2.8$, consistent with this lecture's $O(\log n)$ claim.

Extract-min: `heapify_down`, in code

```
def extract_min(self):
    min_value = self.data[0]
    last = self.data.pop()                  # remove the LAST element
    if self.data:
        self.data[0] = last                  # move it to the root
        self._heapify_down(0)
    return min_value

def _heapify_down(self, i):
    smallest = i
    for child in (self._left_child(i), self._right_child(i)):
        if child < len(self.data) and self.data[child] < self.data[smallest]:
            smallest = child
    if smallest != i:
        self.data[i], self.data[smallest] = self.data[smallest], self.data[i]
        self._heapify_down(smallest)
```

The minimum is always located at index 0 by the heap property, so reading it costs O(1), no search whatsoever — the headline capability this structure is built around. Removing it is more involved: the last element in the array is moved to the root position, which preserves completeness (removing from the end never creates a gap), and then `_heapify_down` bubbles this relocated element downward, at each step swapping with whichever child is smaller, continuing until the heap property is restored or a leaf is reached.

Tracing extract_min completely

```
Before: [0, 3, 1, 7, 4, 5, 2]        extract_min()
Step 1: min_value = 0. pop last (2) → data=[0,3,1,7,4,5]. Move 2 to root:
        [2, 3, 1, 7, 4, 5],  i=0
Step 2: left_child(0)=1 (val 3), right_child(0)=2 (val 1). smallest of {2,3,1} is 1, at index
        [1, 3, 2, 7, 4, 5],  i=2
Step 3: left_child(2)=5 (val 5), right_child(2)=6 (out of bounds, len=6). smallest of {2,5}
```

`Result: [1, 3, 2, 7, 4, 5], returned min_value = 0`

Tracing `extract_min` on the 7-element heap from the previous slide: the minimum, 0, is saved to return. The last element, 2, is popped and moved to the root. `_heapify_down` then compares this relocated 2 against its children (3 and 1), finds 1 smaller, and swaps, moving 2 to index 2. At its new position, 2's only child is index 5 (value 5); since $2 < 5$, no further swap is needed, and the process stops. The result, `[1, 3, 2, 7, 4, 5]`, is a valid heap once again, with the original minimum, 0, correctly returned to the caller.

The picture: bubbling up vs. bubbling down

Both operations move a single element along one root-to-leaf path, exactly like Week 7's BST operations — the direction differs (`heapify_up` moves toward the root, `heapify_down` moves away from it), but both are bounded by the tree's height, which a heap's completeness guarantee keeps at $O(\log n)$ unconditionally.

Why O(1) minimum, but O(n) for anything else

Because the heap property only constrains the vertical relationship between a parent and its children, and says nothing at all about how the left and right subtrees compare to each other, finding an arbitrary value that is not the minimum requires potentially checking every single node — O(n), no better than an entirely unordered array. This is a deliberate and worthwhile trade-off, not a flaw: a heap is optimized narrowly and specifically for repeated "what is the minimum, and remove it" queries, at O(log n) each, and makes no claim to be a general-purpose search structure — that role belongs to Week 7's balanced BST instead.

MTech addition — `heapify`: building a heap in O(n)

```python
def heapify(arr):
    n = len(arr)
    for i in range(n // 2 - 1, -1, -1):     # start from the last NON-leaf node, go backward
        _heapify_down_arr(arr, n, i)
```

Converting an already-existing, arbitrary array into a valid heap can be done naively by inserting all n elements one at a time, each costing $O(\log n)$, for $O(n \log n)$ total. The `heapify` algorithm does better: starting from the last non-leaf node and calling `_heapify_down` on each node in reverse order back to the root, it builds a valid heap in just O(n) total — a genuinely surprising result, since it appears to call an $O(\log n)$ operation n times. The resolution is that most nodes in a complete tree are near the bottom, where `heapify_down` has very little remaining height to bubble through; a careful sum across all levels (rather than a loose per-node bound) shows the true total is $O(n)$, not $O(n \log n)$ — a result worth remembering precisely because the naive per-node bound is misleadingly pessimistic.

Connecting back: the leaderboard case study, resolved differently

It is worth revisiting Week 6 Lecture 4's leaderboard case study one more time. Week 7 Lecture 4 resolved the "maintained ranking structure" gap using a balanced BST, offering full ordering and range-query capability. A max-heap offers a leaner alternative when the actual requirement is narrower than full ranking: O(1) access to the single top-ranked player, and O(log n) updates as scores change. If the real requirement is genuinely just "who is in the top spot" or "process players in priority order," a heap is simpler to implement and often faster in practice than a full balanced BST — but if genuine range queries ("everyone ranked between 100th and 110th") are needed, the BST's extra capability from Week 7 becomes necessary again. Choosing between the two is exactly matching a structure's guarantees to the actual required query pattern, the recurring theme of this course.

Exercise

As an exercise, implement `MinHeap.peek()`, returning the minimum element without removing it, which should cost O(1) by simply reading index 0 directly. Then build a heap from the array `[5, 3, 8, 1, 9, 2]` using both the naive repeated-insertion approach and the `heapify` algorithm from this lecture's MTech addition, and confirm both produce a structure satisfying the heap property — even though the two approaches may arrive at different internal array arrangements, since the heap property does not uniquely determine array layout.

Heapsort: a sorting algorithm, for free (again)

Exactly as Week 7 Lecture 4 derived treesort from BST insertion plus inorder traversal, heapsort falls directly out of this lecture's own operations: build a heap from the input array in O(n) using `heapify`,

then repeatedly call `extract_min` n times, each costing $O(\log n)$, collecting results in increasing order. Total cost: $O(n) + O(n \log n) = O(n \log n)$, matching Week 6's merge sort and quicksort once again, via yet another distinct mechanism. Unlike treesort, heapsort genuinely is used in practice — it is the algorithm introsort (Week 6 Lecture 2's MTech addition) falls back to specifically to guarantee a worst-case bound when quicksort's recursion depth suggests its pathological case is occurring.

Quiz — is this a valid heap?

Given the array `[2, 4, 3, 8, 5, 9]`, determine before checking the answer whether it represents a valid min-heap, checking every parent-child relationship using this lecture's array-index formulas.

Quiz — answer

Checking every parent-child relationship using the array-index formulas: index 0 (value 2) has children at indices 1 (4) and 2 (3), and $2 \leq 4$ and $2 \leq 3$ both hold. Index 1 (value 4) has children at indices 3 (8) and 4 (5), and $4 \leq 8$ and $4 \leq 5$ both hold. Index 2 (value 3) has one child, at index 5 (9), and $3 \leq 9$ holds. Every required relationship is satisfied, confirming this is a valid min-heap — even though, notably, the array is not sorted in any conventional sense (5 appears before 3 in the array, despite being larger), which is precisely the weaker guarantee the heap property provides compared to a fully sorted or BST-ordered structure.

A second worked trace — inserting into a smaller heap

A second complete trace, building a heap from scratch via four insertions: 5, then 3 (which bubbles up past 5), then 8 (which needs no bubbling, already correctly placed relative to its parent), then 1 (which bubbles all the way to the root, past both 5's original position and 3). The final array, `[1, 3, 8, 5]`, is confirmed valid by checking every parent-child relationship directly, exactly the verification method used in this lecture's quiz.

Summary

The heap property requires only that every parent be less than or equal to both of its children, everywhere in the tree — a deliberately weaker guarantee than Week 7's binary search tree property. A heap's guaranteed completeness allows it to be stored entirely in a plain array, with parent and child relationships computed by direct arithmetic rather than followed via pointers. Both `heapify_up` (used for insertion) and `heapify_down` (used for extracting the minimum) cost $O(\log n)$, guaranteed for every heap, since completeness prevents the degeneration risk that plagued Week 7's plain, unbalanced BST. The resulting trade-off — O(1) access to the minimum, but O(n) for finding anything else — is a deliberate design choice suited to a narrow but common class of problems. Next lecture builds priority

queues directly on top of this heap structure and previews Dijkstra's shortest-path algorithm, which Week 12 covers in full using exactly this priority queue.